

Exception and Interrupt HandlingException Handling :

An exception is any condition that needs to halt the normal sequential execution of instructions. Examples are when the ARM core is reset, when an instruction fetch or memory access fails, when an undefined instruction is encountered, when a software interrupt instruction is executed, or when an external interrupt has been raised. Exception handling is the method of processing these exceptions.

Most exceptions have an associated software exception handler - a software routine that executes when an exception occurs. For instance, a Data Abort exception will have a Data Abort handler. The handler first determines the cause of the exception and then services the exception. Servicing takes place either within the handler or by branching to a specific service routine. The Reset exception is a special case since it is used to initialize an embedded system.

① ARM Processor Exceptions and Modes:

Exception	mode	main purpose
Fast Interrupt Request	FIQ	fast interrupt request handling
Interrupt Request	IRQ	interrupt request handling
SWI and Reset	SVC	Protected mode for operating systems
Prefetch Abort and Data Abort	Abort	virtual memory and/or memory protection handling
Undefined Instruction	Undefined	software emulation of hardware coprocessors

Table ①:- ARM Processor exceptions and associated - modes

The Table ①, lists the ARM processor exceptions. Each exceptions causes the core to enter a specific mode. In addition, any of the ARM - processor modes can be entered manually by changing the CPSR.

②

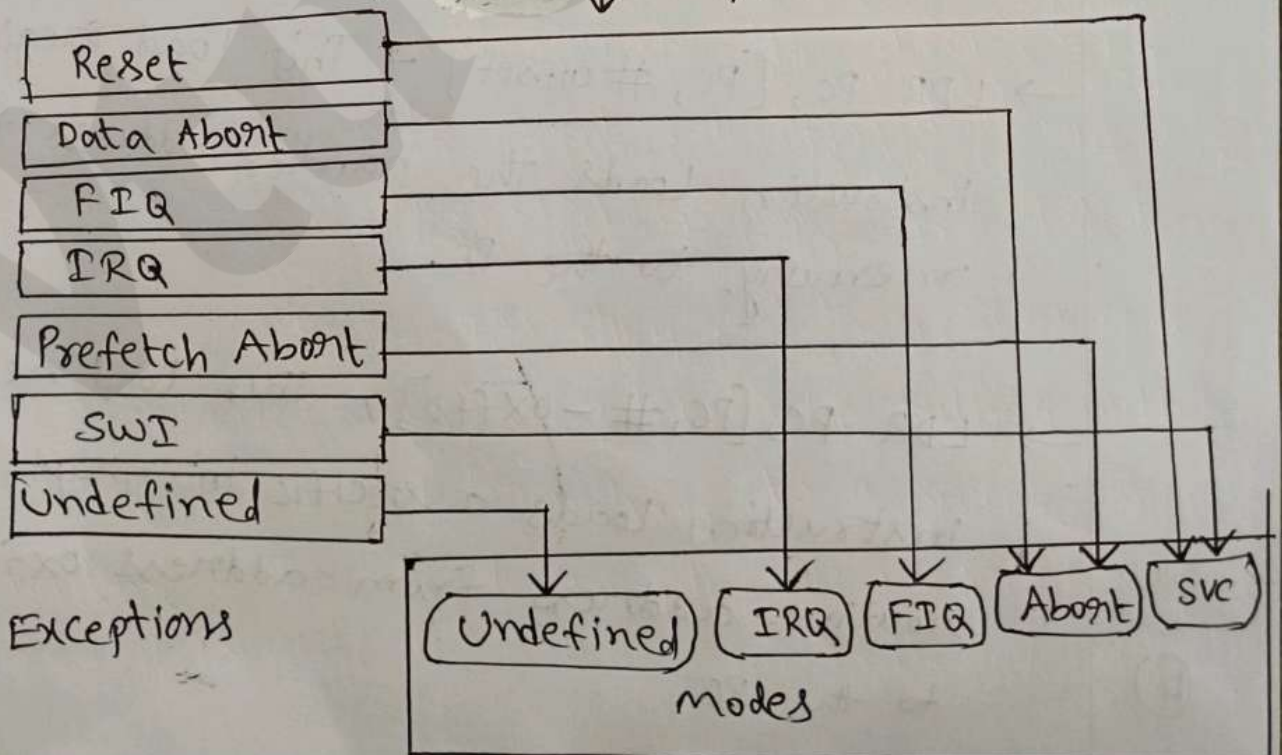
User and System mode are the only two modes that are not entered by a - corresponding exception, in other words, to enter these modes you must modify the CPSR.

When an exception causes a mode change, the core automatically

- Saves the CPSR to the SPSR of the exception-mode
- Saves the PC to the LR of the exception-mode
- Saves the CPSR to the exception mode.
- Sets PC to the address of the exception-handler.

Figure ①

Exceptions and associated modes



The figure ①, shows a simplified view of exceptions and associated modes. Note that when an exception occurs the ARM processor always switches to ARM state.

② Vector Table :

The vector table - a table of addresses that the ARM core branches to when an exception is raised. These addresses commonly contain branch instructions of one of the following forms :-

- $B<address>$ - This Branch instruction provides a branch relative from the PC.
- $LDR\ PC, [PC, \#offset]$ - This load register-instruction loads the handler address from memory to the PC.
- $LDR\ PC, [PC, \#-0xffff0]$ - This load register-instruction loads a specific interrupt service-routine address from address $0xffff030$ to the PC.

→ `mov pc, # immediate` - This move instruction copies an immediate value into the PC.

Exception	mode	Vector Table offset
Reset	SVC	+0X 00
Undefined Inst.	UND	+0X 04
Software Interrupt (SWI)	SVC	+0X 08
Prefetch Abort	ABT	+0X 0C
Data Abort	ABT	+0X 10
Not assigned	—	+0X 14
IRQ	IRQ	+0X 18
FIQ	FIQ	+0X 1C

Table ②: Vector Table and processor modes.

The Table ②, shows the exception, mode, and vector table offset for each exception.

Exception priorities:

Exceptions can occur simultaneously, so the processor has to adopt a priority mechanism. Table 1 shows the various exceptions that occur on the ARM processor and their associated priority level. For instance, the Reset exception is the highest priority and occurs when power is applied to the processor. Thus, when a reset occurs, it takes precedence over all other exceptions. Similarly, when a Data Abort occurs, it takes precedence over all other exceptions apart from a Reset exception. The lowest priority level is shared by two exceptions, the Software Interrupt and Undefined Instruction exceptions. Certain exceptions also disable interrupts by setting the I or F bits in the CPSR, as shown in Table 1.

Each exception is dealt with according to the priority level set out in Table 1.

The following is a summary of the exceptions and how they should be handled, starting with the highest.

⑥

Table 1:

Exceptions	Priority	I bit	Fbit
Reset	1	1	1
Data Abort	2	1	—
Fast Interrupt Request	3	1	1
Interrupt Request	4	1	—
Prefetch Abort	5	1	—
Software Interrupt	6	1	—
Undefined Instruction	6	1	—

Fig1: Exception priority levels

Link Register OFFSETS:

When an exception occurs, the link register is set to a specific address based on the current PC. For instance, when an IRQ exception is raised, the link register lr points to the last executed instruction plus 8. Care has to be taken to make sure the exception handler does not corrupt lr because lr is used to return from an exception handler. The IRQ exception is taken only after the

Current instruction is executed, so the return address has to point to the next instruction, or $lr-4$. Table 1 provides a list of useful address for the different exceptions.

Table 1:

Exception	Address	Use
Reset	—	lr is not defined on a Reset
Data Abort	$lr-8$	points to the instruction that caused the Data Abort exception
FIQ	$lr-4$	return address from FIQ handler
IRQ	$lr-4$	return address from the IRQ handler
Prefetch Abort	$lr-4$	points to the instruction that caused the Prefetch Abort exception
SWI	lr	points to the next instruction after the SWI instruction
Undefined Instruction	lr	points to the next instruction after the undefined instruction

Fig 1: Useful link-register-based addresses.

Interrupts:-

There are two types of interrupts available on the ARM processor. The first type of interrupt causes an exception raised by an external peripheral - namely, IRQ and FIQ. The second type is a specific instruction that causes an exception - the SWI instruction. Both types suspend the normal flow of a program.

→ Assigning interrupts

→ Interrupt latency

→ IRQ and FIQ exceptions

→ Basic interrupt stack design and implementation

Assigning Interrupts

A system designer can decide which hardware peripheral can produce which interrupt request. This decision can be implemented in hardware or software (or both) and depends upon the embedded system being used.

An interrupt controller connects multiple external interrupts to one of the two ARM interrupt requests. Sophisticated controllers can

be programmed to allow an external interrupt source to cause either an IRQ or FIQ exception.

When it comes to assigning interrupts, system designers have adopted a standard design practice:

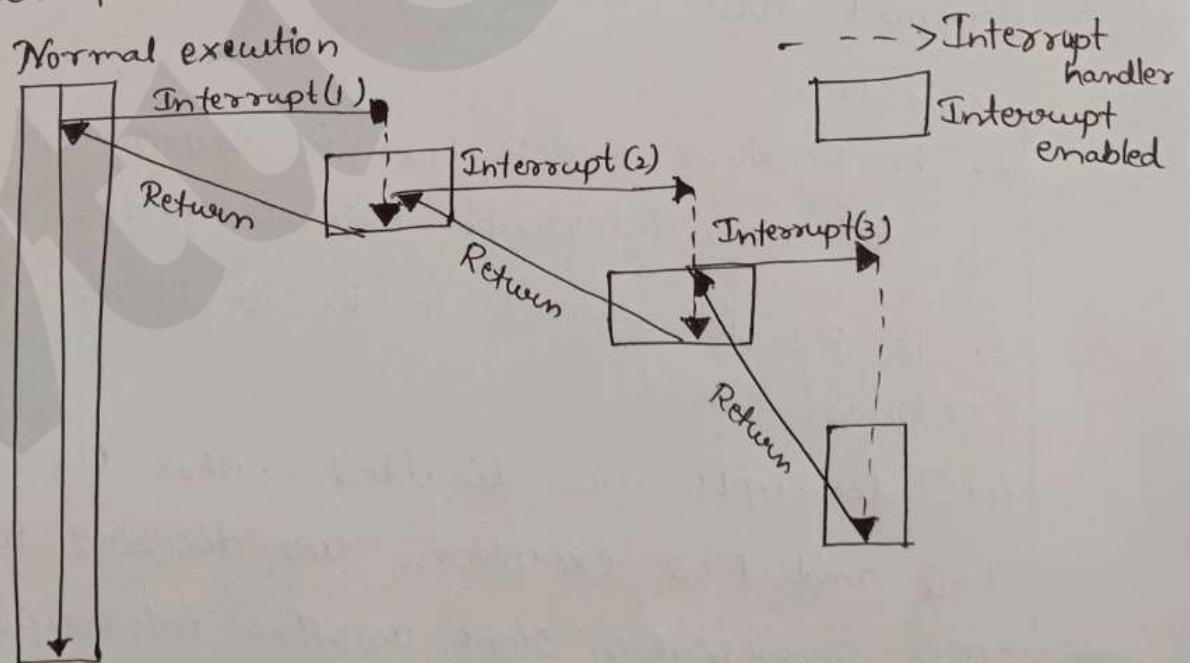
- Software Interrupts are normally reserved to call privileged operating system routines. For example, an SWI instruction can be used to change a program running in user mode to a privileged mode.
- Interrupt Requests are normally assigned for general purpose interrupts.
- Fast Interrupt Requests are normally reserved for a single interrupt source that requires a fast response time.

Interrupt Latency:

Interrupt latency depends on a combination of hardware and software. System architects must balance the system design to handle multiple simultaneous interrupt sources and minimize interrupt latency.

If the interrupts are not handled in a timely manner, then the system will exhibit slow response times.

Software handlers have two main methods to minimize interrupt latency. The first method is to use a nested interrupt handler, which allows further interrupts to occur even when currently servicing an existing interrupt. This is achieved by reenabling the interrupts as soon as the interrupt source has been serviced (so it won't generate more interrupts) but before the interrupt handling is complete. Once a nested interrupt has been serviced, then control is relinquished to the original interrupt service routine.



(11)

Fig1: A three-level nested interrupt

The second method involves prioritization.

You program the interrupt controller to ignore interrupts of the same or lower priority than the interrupt you are handling, so only a higher-priority task can interrupt your handler. You then reenable interrupts.

IRQ and FIQ Exceptions:

IRQ and FIQ exceptions only occur when a specific interrupt mask is cleared in the CPSR.

An IRQ or FIQ exception causes the processor hardware to go through a standard procedure (provided the interrupts are not masked):

1. The processor changes to a specific interrupt request mode, which reflects the interrupt being raised.
2. The previous mode's CPSR is saved into the SPSR of the new interrupt request mode.
3. The PC is saved in the LR of the new interrupt request mode.
4. Interrupts are disabled - either the IRQ or both IRQ and FIQ exceptions are disabled in the CPSR. This immediately stops another interrupt request of the same type being raised.

5. The processor branches to a specific entry in the vector table.

The procedure varies slightly depending upon the type of interrupt being raised. We will illustrate both interrupts with an example.

The first example shows what happens when an IRQ exception is raised, and the second example shows what happens when an FIQ exception is raised.

Example 1: Figure 1. shows what happens when an IRQ exception is raised when the processor is in user mode. The processor starts in state 1. In this example both the IRQ and FIQ exception bits in the CPSR are enabled.

When an IRQ occurs the processor moves into state 2. This transition automatically sets the IRQ bit to one, disabling any further IRQ exceptions. The FIQ exception, however, remains enabled because FIQ has a higher priority and therefore does not get disabled with a low-priority IRQ exception is raised. The CPSR processor mode changes to IRQ mode. The user mode CPSR is automatically copied into SPSR-IRQ.

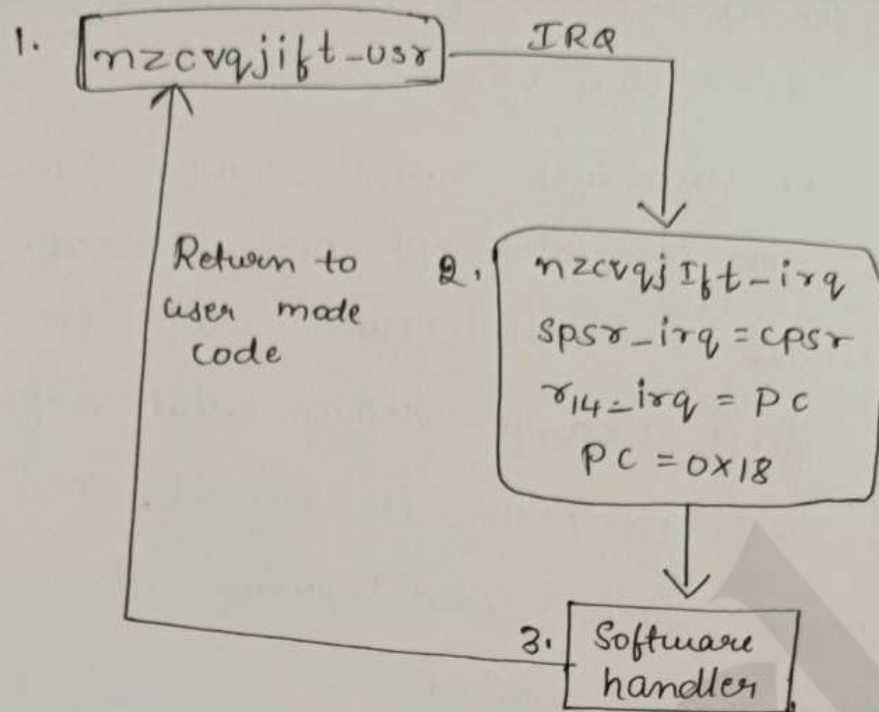


Fig1. Interrupt Request (IRQ)

Register `r14_irq` is assigned the value of the `pc` when the interrupt was raised. The `pc` is then set to the `IRQ` entry `+0x18` in the vector table.

In state 3 the software handler takes over and calls the appropriate interrupt service routine to service the source of the interrupt. Upon completion, the processor mode reverts back to the original user mode code in state 1.

Example 2 Figure 2 shows an example of an FIQ exception. The processor goes through a similar from occurring, procedure as with the IRQ exception, but instead of just masking further IRQ exceptions from occurring, the processor also masks out further FIQ exceptions. This means that both interrupts are disabled when entering the software handler in state 3.

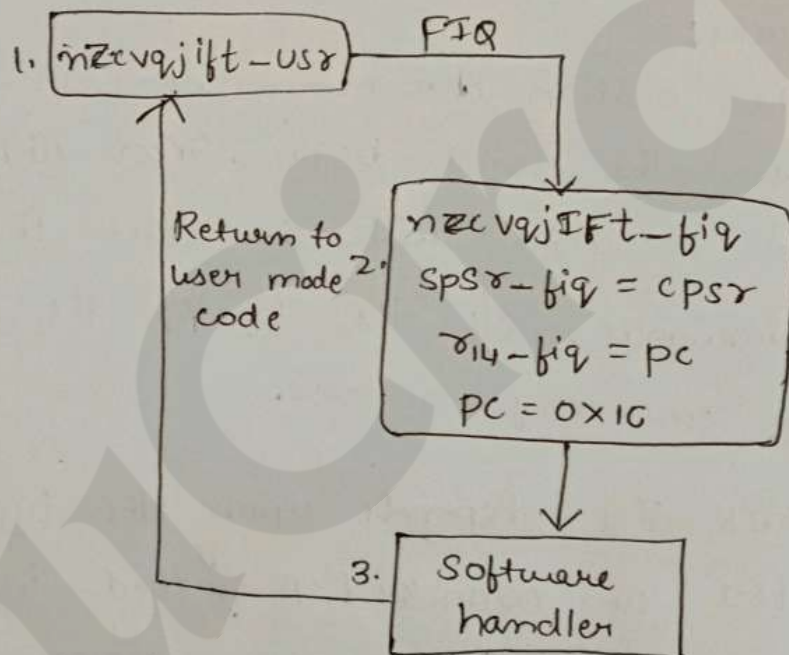


Fig 2. Fast Interrupt Request (FIQ)

Basic Interrupt Stack Design and Implementation:

The design of the exception stacks depends upon these factors:

- Operating system requirements - Each operating system has its own requirements for stack design.
- Target hardware - The target hardware provides a physical limit to the size and positioning of the stack in memory.

Two design decisions need to be made for the stacks:

- The location determines where in the memory map the stack begins. Most ARM-based systems were designed with a stack that descends downwards, with the top of the stack at a high memory address.
- Stack size depends upon the type of handler, nested or nonnested. A nested interrupt handler requires more memory space since the stack will grow with the number of nested interrupts.

A good stack design tries to avoid stack overflow - where the stack extends beyond the allocated memory - because it causes instability in embedded systems.

There are software techniques that identify overflow and that allow corrective measures to take place to repair the stack before irreparable memory corruption occurs. The two main methods are (1) to use memory protection and (2) to call a stack check function at the start of each routine.

Figure 1. shows two typical memory layouts in a linear address space. The first layout, A, shows a traditional stack layout with the interrupt stack stored underneath the code segment. The second layout, B, shows the interrupt stack at the top of the memory above the user stack. The main advantage of layout B over A is that B does not corrupt the vector table when a stack overflow occurs, and so the system has a chance to correct itself when an ~~overflow~~ overflow has been identified.

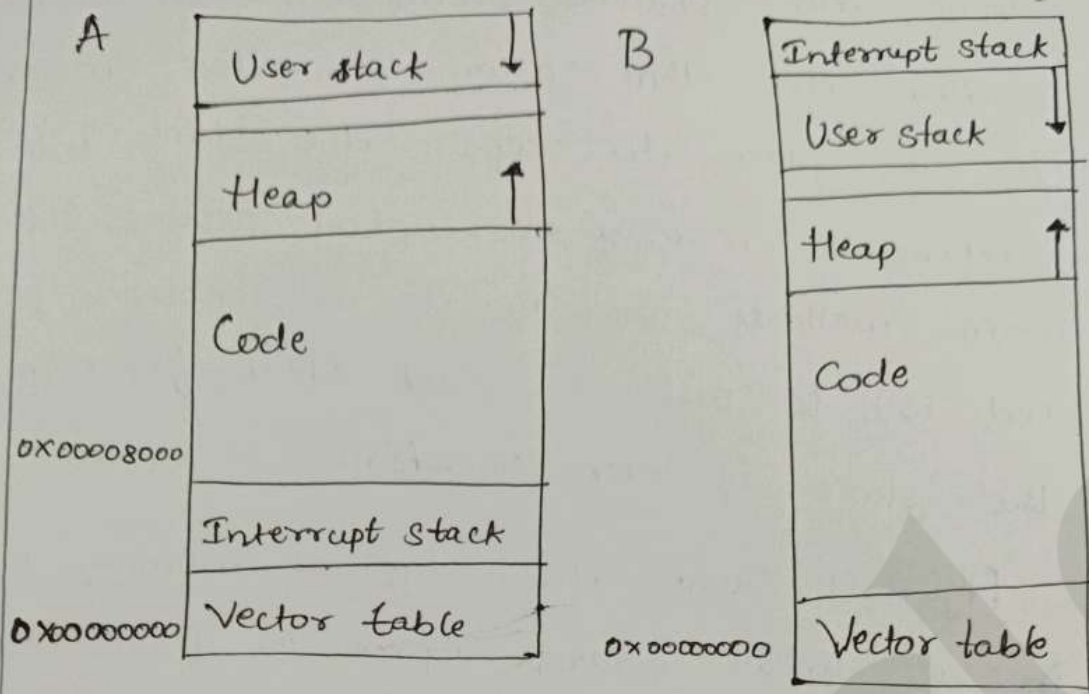


Fig 1: Typical memory layouts

Firmware and Bootloader :-

We realize that the use of terms may differ among engineers, but we will use the following definitions:

- The firmware is the deeply embedded, low level software that provides an interface between the hardware and the application/operating system level software. It resides in the ROM and executes when power is applied to the embedded hardware system. Firmware can remain active after system initialization and supports basic system operations. The choice of which firmware

to use for a particular ARM-based system depends upon the specific application, which can range from loading and executing a sophisticated operating system to simply relinquishing control to a small microkernel.

- The bootloader is a small application that installs the operating system or application onto a hardware target. The bootloader only exists up to the point that the operating system or application is executing, and it is commonly incorporated into the firmware.

To help understand the features of different firmware implementations, we have a common execution flow (see Table 1). Each stage is now discussed in more detail.

The first stage is to set up the target platform - in other words, prepare the environment to boot an operating system since an operating system expects a particular type of environment before it can operate. This step involves making sure that the platform is correctly initialized.

Table 1.

Stage	Features
Set up target platform	Program the hardware system registers Platform identification Diagnostics Debug interface
Abstract the hardware	Command line interpreter Hardware Abstraction layer Device driver
Load a bootable image	Basic filing system
Relinquish control	Alter the pc to point into the new image

Fig1: Firmware execution flow

Diagnostics software provides a useful way for quickly identifying basic hardware malfunctions. Because of the nature of this type of software, it tends to be specific to a particular piece of hardware.

Debug capability is provided in the form of a module or monitor that provides software assistance for debugging code running on a hardware target. This assistance includes the following:

- Setting up breakpoints in RAM. A breakpoint allows a program to be interrupted and the state of the processor core to be examined.

- Listing and modifying memory (using peek and poke operations).
- Showing current processor register contents
- Disassembling memory into ARM and Thumb instruction mnemonics.

ARM Firmware Suite (AFS) :-

ARM has developed a firmware package called the ARM Firmware Suite (AFS). AFS is designed purely for ARM-based embedded system. It provides support for a number of boards and processors including the Intel XScale and StrongARM processors. The package includes two major pieces of technology, a Hardware Abstraction Layer called μ HAL (pronounced micro-HAL) and a debug monitor called Angel.

μ HAL provides a low level device driver framework that allows it to operate over different communication devices (for example, USB, Ethernet, or serial). It also provides a standard API.

Consequently, when a port takes place, the various hardware specific parts must be implemented in accordance with the various μ HAL API functions.

μHAL Supports these main features:

- System initialization — setting up the target platform and processor core. Depending upon the complexity of the platform, this can either be a simple or complicated task.
- Polled serial driver — used to provide a basic method of communication with a host.
- LED support — allows control over the LEDs for simple user feedback. This provides an application the ability to display operational status.
- Timer support — allows a periodic interrupt to be set up. This is essential for preemptive context switching operating systems that require this mechanism.
- Interrupt controllers — support for different interrupt controllers.

The boot monitor in μHAL contains a CHI.

The second technology, Angel, allows communication between a host debugger and a target platform. It allows you to inspect and modify memory, download and execute images, set breakpoints, and display processor register contents. All this control is through the host debugger. The Angel debug monitor must have access to the SWI and IRQ or FIQ vectors.

Angel uses SWI instructions to provide a set of APIs that allows a program to open, read and write to a host filing system.

IRQ/FIQ interrupts are used for communication purpose with the host debugger.

Red HAT REDBOOT

RedBoot is a firmware tool developed by RedHat. It is provided under an Open Source license with no royalties or up front fees. Redboot is designed to execute on different CPUs (for instance, ARM, MIPS, SH and so on). It provides both debug capability through GNU Debugger (GDB), as well as bootloader. The RedBoot software core is based on a HAL. Redboot supports these main features:

- Communication — configuration is over serial or Ethernet. For serial, X-Modem protocol is used to communicate with the GNU Debugger (GDB). For Ethernet, TCP is used to communicate with GDB. Redboot supports a range of network standards, such as bootp, telnet, and tftp.
- Flash ROM memory management — provides a set of filing system routines that can download, update, and erase image in flash ROM. In addition, the images can either be compressed or uncompressed.
- Full operating system support. — supports the loading and booting of Embedded Linux, Red Hat eCos, and many other popular operating systems.

For Embedded Linux, RedBoot supports the ability to define parameters that are passed directly to the kernel upon booting.

Example: Sandstone

We have designed sandstone to be a minimal system. It carries out only the following tasks: set up target platform environment, load a bootable image into memory, and relinquish control to an operating system. It is, however, still a real working example.

The implementation is specific to the ARM Evaluator -7T platform, which includes an ARM7TDMI processor. This example shows you exactly how a simple platform can be set up and a software payload can be loaded into memory and booted. The payload can either be an application or operating system image. Sandstone is a static design and cannot be configured after the build process is complete. Table 1 lists the basic characteristics of Sandstone.

Sandstone Directory layout:-

Sandstone can be found on our web site. If you take a look at Sandstone, you will see that the directory structure is as shown in Fig 1. The structure follows a standard style

Table 1 :

Feature	Configuration
Code	ARM instructions only
Tool chain	ARM Developer Suite 1.2
Image size	700 bytes
Source	17KB
Memory	remapped

Fig 1: Summary of Sandstone

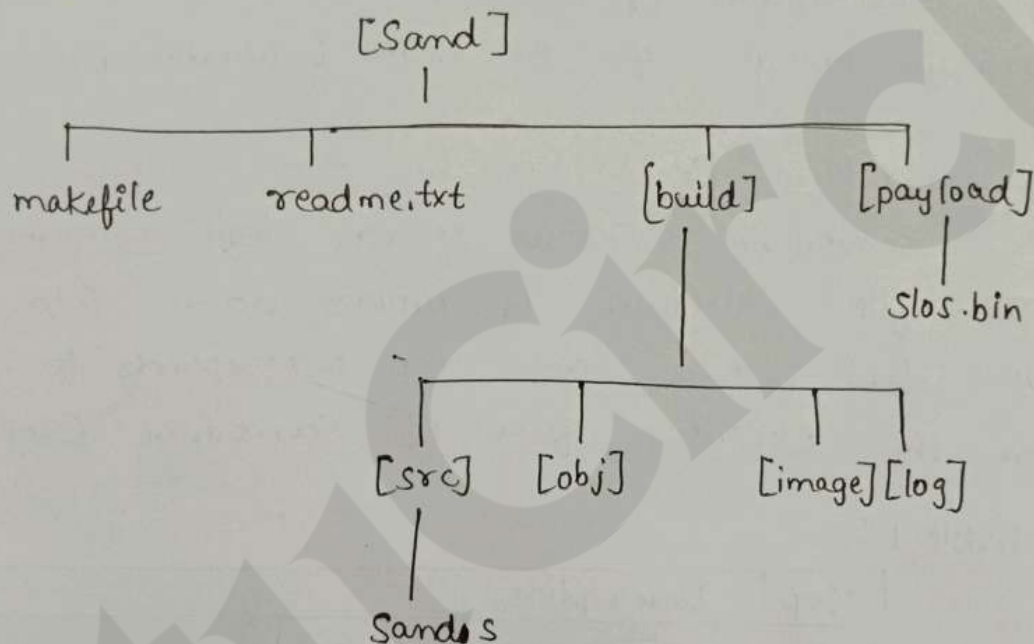


Fig 2: Sandstone directory layout.

that we will continue to use in further chapters. The sandstone source file s and .s is located under the sand/build/src directory.

The Object file produced by the assembler is placed under the build/obj directory. The object file is then linked, and the final Sandstone Image is placed under the sand/build/image directory.

This image includes both the Sandstone code and the payload. The payload image, the image that is loaded and booted by Sandstone, is found under the sand/payload directory.

For information about the Sandstone build procedure, take a look at the `readme.txt` file under the sand directory. This file contains a description of how to build the example binary image for the ARM Evaluator-7T.

Sandstone Code Structure :-

Sandstone consists of a single assembly file. The file structure is broken down into a number of steps, where each step corresponds to a stage in the execution flow of Sandstone (see Table 1)

Table 1:

Step	Description
1	Take the Reset exception
2	Start initializing the hardware
3	Remap memory
4	Initialize communication hardware
5	Bootloader - copy payload and relinquish control

Fig 1: Sandstone execution flow

We will take you through these steps, trying to avoid as much as possible the platform specific parts. You should note that some specific parts are unavoidable (for example, configuring system registers and memory remapping).

The initial goal of Sandstone is to set up the target platform environment so that it can provide some form of feedback to indicate that the firmware is running and has control of the platform.

Step 1: Take the Reset Exception

Execution begins with a Reset exception. Only the reset vector entry is required in the default vector table. It is the very first instruction executed. You can see from the code here that all the vectors, apart from the reset vector, branch to a unique dummy handler - a branch instruction that causes an infinite loop. It is assumed that no exception or interrupt will occur during the operation of Sandstone. The reset vector is used to move the execution flow to the second stage.

AREA start, CODE, READONLY

ENTRY

Sandstone_start

	B	sandstone_initl	; reset vector
	B	ex_und	; undefined vector
	B	ex_swi	; swi vector
	B	ex_pabt	; prefetch abort vector
	B	ex_dabt	; data abort vector
	NOP		; not used ---
	B	int_irq	; irq vector
	B	int_fiq	; fiq vector
ex_und	B	ex_und	; loop forever
ex_swi	B	ex_swi	; loop forever
ex_dabt	B	ex_dabt	; loop forever
ex_pabt	B	ex_pabt	; loop forever
int_irq	B	int_irq	; loop forever
int_fiq	B	int_fiq	; loop forever

Sandstone_start is located at address 0x00000000.

The results of executing step1 are the following:

- Dummy handlers are set up.
- Control is passed to code to initialize the hardware.

Step 2: Start Initializing the Hardware

The primary phase in initializing hardware is setting up system registers. These registers have to be set up before accessing the hardware.

For example, the ARM Evaluator-7T has a seven-segment display, which we have chosen to be used as a feedback tool to indicate that the firmware is active. Before we can set up the segment display, we have to position the base address of the system registers to a known location. In this case, we have picked the default address $0x03ff0000$, since this places all the hardware system registers away from both ROM and RAM, separating the peripherals and memory.

Consequently, all the microcontroller memory-mapped registers are located as an offset from $0x03ff0000$. This is achieved using the following code:

`sandstone_init1`

`LDR r3, =SYSCFG ; where SYSCFG = 0x03ff0000`

`LDR r4, =0x03ff0fa0`

`STR r4, [r3]`

Register $r3$ contains the default system register base address and is used to set the new default address, as well as other specific attributes such as the cache.

Register π_4 contains the new configuration. The top 16 bits contain the high address of the new system register base address $0x03ff$, and the lower 16 bits contain the new attribute settings $0xffa0$.

After setting up the system register base address, the segment display can be configured. The segment display hardware is used to show Sandstone's progress. Note that the segment display is not shown since it is hardware specific.

The results of executing step 2 are the following:

- The system registers are set from a known base address - $0x03ff0000$.
- The segment display is configured, so that it can be used to display progress.

Step 3: Remap Memory

One of the major activities of hardware initialization is to set up the memory environment. Sandstone is designed to initialize SRAM and remap memory. This process occurs fairly early on in the initialization of the system. The platform starts in a known memory state, as shown in Table 1.

As you can see, when the platform is powered up, only flash ROM is assigned a location in the memory map. The two SRAM banks (0 and 1) have not been initialized and are not available.

The two SRAM banks (0 and 1) have not been initialized and are not available. The next stage is to bring in the two SRAM banks and remap flash ROM to a new location. This is achieved using the following code.

```
LDR r14, =sandstone_init2
```

```
LDR r4, =0x01800000 ; new flash ROM location
```

Table 1

Memory type	Start address	End address	Size
Flash ROM	0x00000000	0x00080000	512 k
SRAM bank 0	Unavailable	unavailable	256 k
SRAM bank 1	Unavailable	unavailable	256 k

Fig 1: Initial memory state

```
ADD r14, r14, r4
```

```
ADRL r0, memorymaptable_str
```

```
LDMIA r0, {r1-r12}
```

```
LDR r0, =EXTDBWTH ; = (SXSCFG + 0x3010)
```

```
STMIA r0, {r1-r12}
```

```
MOV PC, r14 ; jump to remapped memory
```

sandstone_init2

; Code after sandstone_init2 executes @ +0x1800000

The first part of the code calculates the absolute address of the routine `sandstone-init2` before remapping takes place. This address is used by `sandstone` to jump to the next routine in the new remapped environment.

The second part carries out the memory remapping. The new memory map data is loaded into registers r_1 to r_{12} , from a structure pointed by `memorymaptable_str`. This structure, using the registers, is then written to the memory controller offset `0x3010` from system configuration register. Once this is complete, the new memory map as shown in

Table 1. is active

You can see that the SRAM banks are now available, and the flash ROM is set to a higher address. The final part is to jump to the next routine, or stage, of the firmware.

This jump is achieved by taking advantage of the ARM pipeline. Even though the new memory environment is active, the next instruction has already been loaded into the pipeline. The next routine can be called by moving the contents of register r_{14} (the address `sandstone-init2`) into the PC. We achieve this by using a single `MOV` instruction that follows immediately after the `semap` code.

Table 1

Type	Start address	End address	Size
Flash ROM	0x01800000	0x01880000	512k
SRAM bank 0	0x00000000	0x00040000	256k
SRAM bank 1	0x00040000	0x00080000	256k

Fig 1: Remapping.

The results of executing steps are the following

- Memory has been remapped as shown in Table 1
- pc now points to the next step. This address is located in the newly remapped flash ROM.

Step 4: Initialize Communication Hardware:-

Communication initialization involves configuring a serial port and outputting a standard banner. The banner is used to show that the firmware is fully functional and memory has been successfully remapped. Again, because the code for initializing the serial port on the ARM Evaluator-7T is hardware specific, it is not shown.

The serial port is set to 9600 baud, no parity, one stop bit, and no flow control. If a serial cable is attached to the board, then the host terminal has to be configured with these settings.

The results of executing step 4 are the following:

- Serial port initialized - 9600 baud, no parity, one stop bit, and no flow control
- Sandstone banner sent out through the serial port;

sandstone Firmware (0.01)

-platform e7t

-status alive

-memory remapped

+booting payload

Step 5: Bootloader - Copy Payload and Relinquish Control

The final stage involves copying a payload and relinquishing control of the pc over to the copied payload. This is achieved using the code shown here.

The first part of the code sets up the registers r_{12} , r_{13} and r_{14} used in the block copy. The bootloader code assumes that the payload is a plain binary image that requires no deciphering or uncompressing.

Sandstone-load-and-boot

MOV r_{13} , #0 ; destination addr

LDR r_{12} , payload-start-address ; start addr

LDR r_{14} , payload-end-address ; end addr

-copy

LDMIA r_{12} !, { r_0 - r_{11} }

STMIA r_{13} !, { r_0 - r_{11} }

CMP r_{12} , r_{14}

BLT copy

MOV PC, #0

payload-start-address

DCD startAddress

payload-end-address

DCD endAddress

Destination register r_{13} points to the beginning of SRAM, in this case $0x00000000$.

The source register r_{12} points to the start of the payload, and the source and register r_{14} points to the end of the payload. Using these registers, the payload is then copied into SRAM.

Control of the pc is then relinquished to the payload by forcing the pc to the entry address of the copied payload. For this particular payload the entry point is address $0x00000000$. The payload now has control of the system.

The result of executing step 5 are the following:

- Payload copied into SRAM, address $0x00000000$
- Control of the pc is relinquished to the payload;
 $PC = 0x00000000$.
- The system is completely booted. The following output is shown on the serial port:

Sandstone Firmware (0.01) .

- platform e7t
- status - alive
- memory remapped

+ booting payload . . .

Simple Little OS (0.09)

- initialized ok
- running on e7t.

e7t: